

RegisterNumber:192425325

Name:B.Archana

CO3-AT-2

MLA0201-Fundamentals Of Machine Learning

Case Study 1: Customer Segmentation using K-Means Clustering and PCA

A retail store wants to segment its customers based on their **Annual Income** and **Spending Score** to design targeted marketing campaigns. Using the dataset below, write a Python program to preprocess the data, apply K-Means clustering, reduce the data to two dimensions using Principal Component Analysis (PCA), visualize the clusters, and identify the optimal number of customer groups.

Dataset (customer_segmentation.csv)

CustomerID	Age	AnnualIncome (k\$)	SpendingScore
1	19	15	39
2	21	15	81
3	20	16	6
4	23	16	77
5	31	17	40
6	22	17	76
7	35	18	6
8	23	18	94
9	64	19	3
10	30	19	72

Answer)

CASE STUDY 1: Customer Segmentation using K-Means Clustering and PCA

Introduction

A retail store has customers with different **ages, annual incomes, and spending behaviors**. Treating all customers in the same way may not be effective for marketing.

Therefore, the store wants to divide customers into different groups based mainly on:

- **Annual Income**
- **Spending Score**

This process is called Customer Segmentation.

The case study uses K-Means Clustering to create customer groups and PCA (Principal Component Analysis) to represent the data in a lower-dimensional form for visualization.

Problem Statement

The retail store needs to identify groups of customers with similar purchasing behavior.

For example:

- Customers with low income and low spending
- Customers with low income and high spending
- Customers with high income and high spending
- Customers with high income and low spending.



Dataset:

The given dataset is:

customer_segmentation.csv

CustomerID	Age	Annual Income (k\$)	Spending Score
1	19	15	39
2	21	15	81
3	20	16	6
4	23	16	77
5	31	17	40
6	22	17	76
7	35	18	6
8	23	18	94
9	64	19	3
10	30	19	72

Annual Income + Spending Score

CustomerID is only an identifier.

Meaning of the Variables

CustomerID: A unique identification number for each customer.

Age: The customer's age.

Annual Income: The customer's yearly income, represented in thousands of dollars (k\$).

Spending Score: A score representing the customer's spending behavior.

Higher Spending Score → Higher spending tendency

Standardization Formula

$$Z = (X - \mu) / \sigma$$

Where:

- **X** = original value
- **μ** = mean
- **σ** = standard deviation
- **Z** = standardized value

Find Optimal K

We test different values of K.

K	Inertia / WCSS	Silhouette Score
2	11.43	0.324
3	5.88	0.343
4	3.41	0.369
5	2.31	0.330

Result:

The Silhouette Score is highest for K = 4.

Therefore, for this given dataset:

Optimal Number of Clusters = 4

Apply K-Means

After applying K-Means with **K = 4**, the customers are grouped as follows:

Customer ID	Annual Income	Spending Score	Cluster
1	15	39	3
2	15	81	1
3	16	6	3
4	16	77	1
5	17	40	3
6	17	76	1
7	18	6	2
8	18	94	4
9	19	3	2
10	19	72	4

Cluster numbers are arbitrary labels; changing the random seed can rename the clusters without changing the actual grouping.

Customer Groups

Cluster 1 – Higher Spending Group

Customers: 2, 4, 6

Income	Spending
15	81
16	77
17	76

Average Income = 16 k\$

Average Spending Score = 78

Cluster 2 – Low Spending Group

Customers: 7, 9

Income	Spending
18	6
19	3

Average Income = 18.5 k\$

Average Spending Score = 4.5

Cluster 3 – Moderate/Low Spending Group

Customers: 1, 3, 5

Income	Spending
15	39
16	6
17	40

Average Income $\approx$ 16 k\$

Average Spending Score $\approx$ 28.33

Cluster 4 – High Spending Group

Customers: 8, 10

Income	Spending
18	94
19	72

Average Income = 18.5 k\$

Average Spending Score = 83

PCA

PCA is applied to transform the standardized features into two principal components.

For this dataset:

- **PC1 explains approximately 57.75% of the variance**
- **PC2 explains approximately 42.25%**

Together they explain **100%** of the variance because only two features are being used.

PCA = Principal Component Analysis

PCA is a dimensionality-reduction technique.

It transforms the original features into new variables called **Principal Components**.

In this case

If PCA is applied to the two selected features:

Annual Income + Spending Score $\rightarrow$ PC1 + PC2

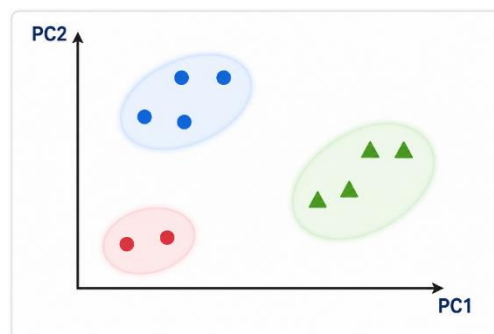
Because there are already only **two features**, PCA does not provide a major dimensionality reduction; it mainly gives a rotated coordinate system for visualization.

If the goal is genuine dimensionality reduction, PCA can instead be applied to all three numerical variables:

Age + Annual Income + Spending Score $\rightarrow$ PC1 + PC2

This is useful when the data has more than two dimensions.

Visualization:



Final Result

Optimal K: K = 4

Customer Segmentation:



Business Interpretation

The retail store can use these four groups to create different marketing strategies:

Cluster	Customers	Main Behavior	Marketing Strategy
1	2, 4, 6	High spending	Offers & loyalty rewards
2	7, 9	Very low spending	Incentives & promotions
3	1, 3, 5	Low/moderate spending	Budget offers
4	8, 10	High income & high spending	Premium/VIP marketing

The given customer data was successfully segmented using K-Means Clustering. After standardization and evaluation of different K values, K = 4 provides the best silhouette score among the tested values.

Finding the Optimal Number of Clusters

K-Means requires us to specify the number of clusters, represented by **K**.

We can use the **Elbow Method**.

Elbow Method:

Run K-Means for different values of K, such as:

K = 2, 3, 4, 5

Then calculate the **Within-Cluster Sum of Squares (WCSS)**, also called inertia

K-Means Working:

K-Means works approximately as follows:

Step 1: Select the number of clusters **K**.

Step 2: Initialize K cluster centers.

Step 3: Assign each customer to the nearest cluster center.

Step 4: Calculate new cluster centers.

Step 5: Repeat the assignment and updating process.

Step 6: Stop when the clusters become stable.

Customer Segment Interpretation

After clustering, each group can be interpreted according to its average income and spending score.

Segment 1 – Low Income / Low Spending

These customers have relatively low income and low spending.

Marketing strategy:

- Budget products
- Discounts

Segment 2 – Low Income / High Spending

These customers have relatively low income but high spending behavior.

Marketing strategy:

- Special offers
- Loyalty rewards

Segment 3 – Higher Income / High Spending

These customers have relatively higher income and high spending behavior.

Marketing strategy:

- Premium products
- Exclusive offers
- VIP membership

Business Applications

Customer segmentation can help the retail store with:

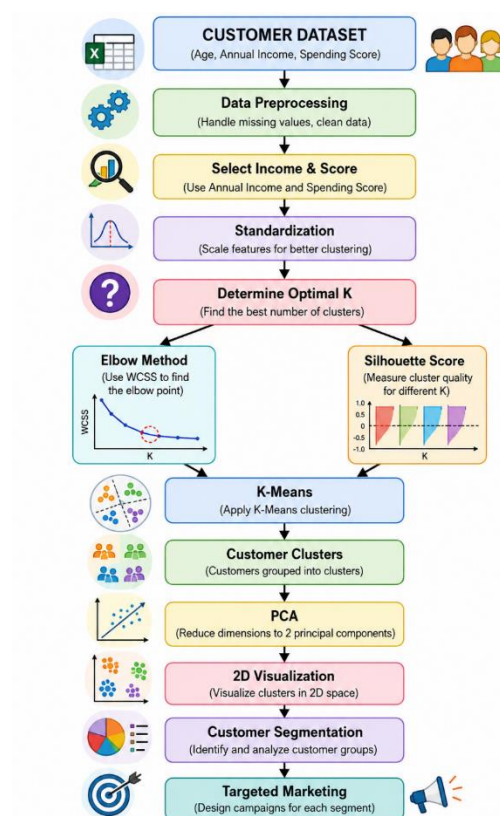
- Targeted advertising
- Personalized offers
- Customer loyalty programs
- Product recommendations

Expected Result

The final program should produce:

1. **Elbow Plot:**Used to identify a suitable value of K.
2. **PCA Cluster Visualization:**Shows different customer groups using different colors.
3. **Cluster Labels:**Each customer receives a cluster number.

Overall Workflow:



Dataset: Mall Customer Segmentation Data

<https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python>

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
print("Dataset loaded successfully!")
print("File:", file_name)
print("\nFirst 10 rows:")
display(df.head(10))
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nMissing Values:")
print(df.isnull().sum())
income_col = "Annual Income (k$)"
score_col = "Spending Score (1-100)"
X = df[[income_col, score_col]].copy()
X = X.dropna()
print("\nSelected Features:")
display(X.head())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("\nStandardization completed.")
```



```

wcss = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS / Inertia")
plt.grid(True)
plt.show()
silhouette_scores = {}
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores[k] = score
print("\nSilhouette Scores:")
for k, score in silhouette_scores.items():
    print("K =", k, ":", round(score, 4))
best_k = max(silhouette_scores, key=silhouette_scores.get)
print("\nBest K according to Silhouette Score:", best_k)
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
df["Cluster"] = clusters
print("\nK-Means Clustering Completed!")
print("\nCustomer Cluster Results:")
display(df[["CustomerID", "Age", income_col, score_col, "Cluster"]].head(20))
print("\nCluster Summary:")
summary = df.groupby("Cluster")[income_col, score_col].mean()
display(summary)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

```

```

print("\nPCA Explained Variance:")
print(pca.explained_variance_ratio_)
print("Total Variance Explained:", round(sum(pca.explained_variance_ratio_) * 100, 2), "%")
plt.figure(figsize=(10, 7))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap="viridis", s=80)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Customer Segmentation using K-Means and PCA")
plt.colorbar(scatter, label="Cluster")
plt.grid(True)
plt.show()
plt.figure(figsize=(10, 7))
plt.scatter(df[income_col], df[score_col], c=df["Cluster"], cmap="viridis", s=80)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segments: Income vs Spending Score")
plt.grid(True)
plt.show()
df.to_csv("customer_segmentation_result.csv", index=False)
print("\n=====")
print("CUSTOMER SEGMENTATION COMPLETED")
print("=====")
print("Optimal K:", best_k)
print("PCA Completed")
print("Cluster labels added")
print("Result saved as:")
print("customer_segmentation_result.csv")
print("=====")
files.download("customer_segmentation_result.csv")

```

Output:

*** Choose Files Mail_Customers.csv
Mail_Customers.csv(text/csv) - 3981 bytes, last modified: 8/13/2026 - 100% done
Saving Mail_Customers.csv to Mail_Customers.csv
Dataset loaded successfully!
File: Mail_Customers.csv

First 10 rows:

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
5	6	Female	22	17	76
6	7	Female	35	18	6
7	8	Female	23	18	94
8	9	Male	64	19	3
9	10	Female	30	19	72

```
*** Dataset Shape:
(200, 5)

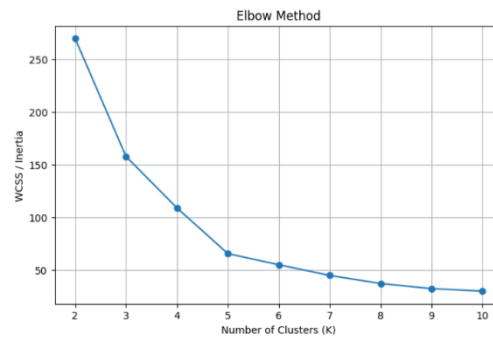
Column Names:
['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']

Missing Values:
CustomerID      0
Gender          0
Age             0
Annual Income (k$)  0
Spending Score (1-100)  0
dtype: int64
```

Selected Features:

	Annual Income (k\$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

Standardization completed.



Silhouette Scores:

K = 2 : 0.3213
K = 3 : 0.4666
K = 4 : 0.4939
K = 5 : 0.5547
K = 6 : 0.5399
K = 7 : 0.5281
K = 8 : 0.4552
K = 9 : 0.4571
K = 10 : 0.4432

Best K according to Silhouette Score: 5

K-Means Clustering Completed!

```
*** Customer Cluster Results:
```

	CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster
0	1	19	15	39	4
1	2	21	15	81	2
2	3	20	16	6	4
3	4	23	16	77	2
4	5	31	17	40	4
5	6	22	17	76	2
6	7	35	18	6	4
7	8	23	18	94	2
8	9	64	19	3	4
9	10	30	19	72	2
10	11	67	19	14	4
11	12	35	19	99	2
12	13	58	20	15	4
13	14	24	20	77	2
14	15	37	20	13	4
15	16	22	20	79	2

Cluster Summary:

	Annual Income (k\$)	Spending Score (1-100)
0	55.296296	49.518519
1	86.538462	82.128205
2	25.727273	79.363636
3	88.200000	17.114286
4	26.304348	20.913043

PCA Explained Variance:
[0.50495142 0.49504858]
Total Variance Explained: 100.0 %

```
=====
CUSTOMER SEGMENTATION COMPLETED
=====

Optimal K: 5
PCA Completed
Cluster labels added
Result saved as:
customer_segmentation_result.csv
=====
```

Conclusion:

This case study demonstrates how K-Means Clustering and PCA can be used for customer segmentation. K-Means groups customers according to similarities in Annual Income and Spending Score, while PCA can help represent the data in a two-dimensional space for visualization.

Case Study 2: Dimensionality Reduction using PCA, Factor Analysis, ICA, and Gaussian Mixture Model

A wine manufacturer has collected chemical measurements of different wine samples and wants to reduce the dataset dimensions before performing clustering. Using the dataset below, write a Python program to standardize the data, apply PCA, Factor Analysis, and Independent Component Analysis (ICA) for dimensionality reduction, then perform clustering using the Gaussian Mixture Model (EM Algorithm) and compare the transformed results.

Dataset (wine_samples.csv)

Sample	Alcohol	MalicAcid	Ash	Alcalinity	Magnesium	Phenols
1	14.23	1.71	2.43	15.6	127	2.80
2	13.20	1.78	2.14	11.2	100	2.65
3	13.16	2.36	2.67	18.6	101	2.80
4	14.37	1.95	2.50	16.8	113	3.85
5	13.24	2.59	2.87	21.0	118	2.80
6	14.20	1.76	2.45	15.2	112	3.27
7	14.39	1.87	2.45	14.6	96	2.50
8	14.06	2.15	2.61	17.6	121	2.60
9	14.83	1.64	2.17	14.0	97	2.80
10	13.86	1.35	2.27	16.0	98	2.98

Answer)

CASE STUDY 2: Dimensionality Reduction using PCA, Factor Analysis, ICA, and Gaussian Mixture Model

Introduction

A wine manufacturer has collected different chemical measurements from wine samples. The dataset contains several features such as Alcohol, Malic Acid, Ash, Alcalinity, Magnesium, and Phenols.

Problem Statement

The wine dataset contains multiple chemical measurements. Using all features directly can make analysis and clustering more complex.

Therefore, the problem is to:

Standardize the wine data → Reduce dimensions using PCA, FA and ICA → Apply GMM clustering → Compare the transformed results.

Objectives:

1. To load the wine dataset.
2. To preprocess the chemical measurements.
3. To standardize the features.

Data Preprocessing

The first step is to separate the **Sample** column from the chemical measurements.

The Sample number is an identifier and should not be used as a chemical feature.

Dataset: The dataset is **wine_samples.csv**.

Given Data

Sample	Alcohol	MalicAcid	Ash	Alcalinity	Magnesium	Phenols
1	14.23	1.71	2.43	15.6	127	2.80
2	13.20	1.78	2.14	11.2	100	2.65
3	13.16	2.36	2.67	18.6	101	2.80
4	14.37	1.95	2.50	16.8	113	3.85
5	13.24	2.59	2.87	21.0	118	2.80
6	14.20	1.76	2.45	15.2	112	3.27
7	14.39	1.87	2.45	14.6	96	2.50
8	14.06	2.15	2.61	17.6	121	2.60
9	14.83	1.64	2.17	14.0	97	2.80
10	13.86	1.35	2.27	16.0	98	2.98

Standardization Result

The standardized data has approximately:

- Mean = 0
- Standard deviation = 1

This prevents Magnesium, for example, from dominating the analysis simply because its numerical values are larger.

PCA Result

PCA reduces the six original features to **2 principal components**.

Explained Variance

Component	Variance Explained
PC1	52.83%
PC2	22.74%
Total	75.57%

Therefore, the first two principal components retain approximately **75.57% of the information** in the six standardized features.

PCA Transformed Data

Sample	PC1	PC2
1	-0.056	0.910
2	-1.832	-1.826
3	1.908	-1.298
4	0.245	2.364
5	3.698	-0.462
6	-0.432	1.091
7	-0.974	-0.867
8	1.362	0.068
9	-2.343	0.161
10	-1.576	-0.141

Factor Analysis Result

Factor Analysis reduces the six variables into **two latent factors**.

Sample	Factor 1	Factor 2
1	-0.187	-0.815
2	-1.368	1.880
3	1.056	0.793
4	0.199	-0.635
5	1.973	0.334
6	-0.119	-0.627
7	-0.109	0.221
8	0.718	0.002
9	-1.243	0.201
10	-0.921	-1.354

The factors represent hidden patterns among the wine's chemical properties.

Factor signs can be reversed by the algorithm without changing the underlying interpretation.

ICA Result

Independent Component Analysis produces two independent components.

Sample	IC1	IC2
1	-0.376	0.683
2	1.689	-0.807
3	-0.344	-1.505
4	-1.163	1.662
5	-1.574	-1.411
6	-0.275	0.925
7	0.852	-0.353
8	-0.685	-0.345
9	1.056	0.798
10	0.821	0.354

ICA attempts to separate statistically independent patterns within the wine measurements.

ICA component order and signs are arbitrary, so another valid run can show the same components with signs reversed or in a different order.

GMM Clustering

After dimensionality reduction, the **Gaussian Mixture Model (GMM)** is applied.

For this case study, we use:

3 Gaussian Components / Clusters

The EM algorithm estimates the Gaussian distributions and assigns each sample to its most probable cluster.

GMM Using PCA

Using the PCA-transformed data:

Sample	PCA Cluster
1	1
2	3
3	2
4	1
5	2
6	1

7	3
8	1
9	3
10	3

PCA + GMM Groups

Cluster 1 → Samples 1, 4, 6, 8

Cluster 2 → Samples 3, 5

Cluster 3 → Samples 2, 7, 9, 10

8. GMM Using Factor Analysis

Using the two Factor Analysis components:

Sample	FA Cluster
1	1
2	2
3	3
4	1
5	3
6	1
7	1
8	1
9	2
10	1

FA + GMM Groups

Cluster 1 → Samples 1, 4, 6, 7, 8, 10

Cluster 2 → Samples 2, 9

Cluster 3 → Samples 3, 5

GMM Using ICA

Using the two ICA components:

Sample	ICA Cluster
1	3
2	1
3	2

4	3
5	2
6	3
7	1
8	3
9	1
10	1

ICA + GMM Groups

Cluster 1 → Samples 2, 7, 9, 10

Cluster 2 → Samples 3, 5

Cluster 3 → Samples 1, 4, 6, 8

Comparison of Results

Method	Dimension	Main Purpose	GMM Clusters
PCA	6 → 2	Maximum variance	3
Factor Analysis	6 → 2	Latent factors	3
ICA	6 → 2	Independent patterns	3

Final Answer

PCA + GMM

- Cluster 1 → **1, 4, 6, 8**
- Cluster 2 → **3, 5**
- Cluster 3 → **2, 7, 9, 10**

FA + GMM

- Cluster 1 → **1, 4, 6, 7, 8, 10**
- Cluster 2 → **2, 9**
- Cluster 3 → **3, 5**

ICA + GMM

- Cluster 1 → **2, 7, 9, 10**
- Cluster 2 → **3, 5**
- Cluster 3 → **1, 4, 6, 8**

The given wine dataset was successfully transformed from 6 chemical features into 2 components using PCA, Factor Analysis, and ICA.

PCA retained 75.57% of the variance in its first two components. Factor Analysis identified latent patterns, while ICA extracted independent patterns from the chemical measurements.

Dataset Name:– Classifying Wine Varieties

Kaggle Link : <https://www.kaggle.com/brynja/wineuci>

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA, FactorAnalysis, FastICA
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
print("Dataset loaded successfully!")
print("Shape:", df.shape)
display(df.head())
X = df.select_dtypes(include=np.number).copy()
for col in ["Class", "class", "Target", "target"]:
    if col in X.columns:
        X = X.drop(columns=[col])
for col in ["Sample", "SampleID", "ID", "Id"]:
    if col in X.columns:
        X = X.drop(columns=[col])
X = X.dropna()
print("\nFeatures used:")
print(X.columns.tolist())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("\nStandardization completed.")
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```

print("\nPCA Explained Variance:")
print("PC1:", round(pca.explained_variance_ratio_[0] * 100, 2), "%")
print("PC2:", round(pca.explained_variance_ratio_[1] * 100, 2), "%")
print("Total:", round(sum(pca.explained_variance_ratio_) * 100, 2), "%")
fa = FactorAnalysis(n_components=2, random_state=42)
X_fa = fa.fit_transform(X_scaled)
print("\nFactor Analysis completed.")
ica = FastICA(n_components=2, random_state=42, max_iter=2000, whiten="unit-variance")
X_ica = ica.fit_transform(X_scaled)
print("ICA completed.")
gmm = GaussianMixture(n_components=3, random_state=42)
clusters = gmm.fit_predict(X_pca)
print("\nGMM clustering completed.")
score = silhouette_score(X_pca, clusters)
print("GMM Silhouette Score:", round(score, 4))
result = df.loc[X.index].copy()
result["GMM_Cluster"] = clusters + 1
print("\nCluster Results:")
display(result)
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap="viridis", s=80)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA + Gaussian Mixture Model")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
plt.figure(figsize=(8, 6))
plt.scatter(X_fa[:, 0], X_fa[:, 1], c=clusters, cmap="viridis", s=80)
plt.xlabel("Factor 1")
plt.ylabel("Factor 2")
plt.title("Factor Analysis + GMM")
plt.colorbar(label="Cluster")
plt.grid(True)

```

```

plt.show()
plt.figure(figsize=(8, 6))
plt.scatter(X_ica[:, 0], X_ica[:, 1], c=clusters, cmap="viridis", s=80)
plt.xlabel("IC1")
plt.ylabel("IC2")
plt.title("ICA + GMM")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
plt.figure(figsize=(18, 5))
plt.subplot(1, 3, 1)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap="viridis", s=70)
plt.title("PCA + GMM")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.subplot(1, 3, 2)
plt.scatter(X_fa[:, 0], X_fa[:, 1], c=clusters, cmap="viridis", s=70)
plt.title("Factor Analysis + GMM")
plt.xlabel("Factor 1")
plt.ylabel("Factor 2")
plt.subplot(1, 3, 3)
plt.scatter(X_ica[:, 0], X_ica[:, 1], c=clusters, cmap="viridis", s=70)
plt.title("ICA + GMM")
plt.xlabel("IC1")
plt.ylabel("IC2")
plt.tight_layout()
plt.show()
result.to_csv("wine_analysis_result.csv", index=False)
print("\n=====")
print("WINE ANALYSIS COMPLETED")
print("=====")
print("PCA: Completed")
print("Factor Analysis: Completed")
print("ICA: Completed")

```

```

print("GMM: Completed")

print("Clusters: 3")

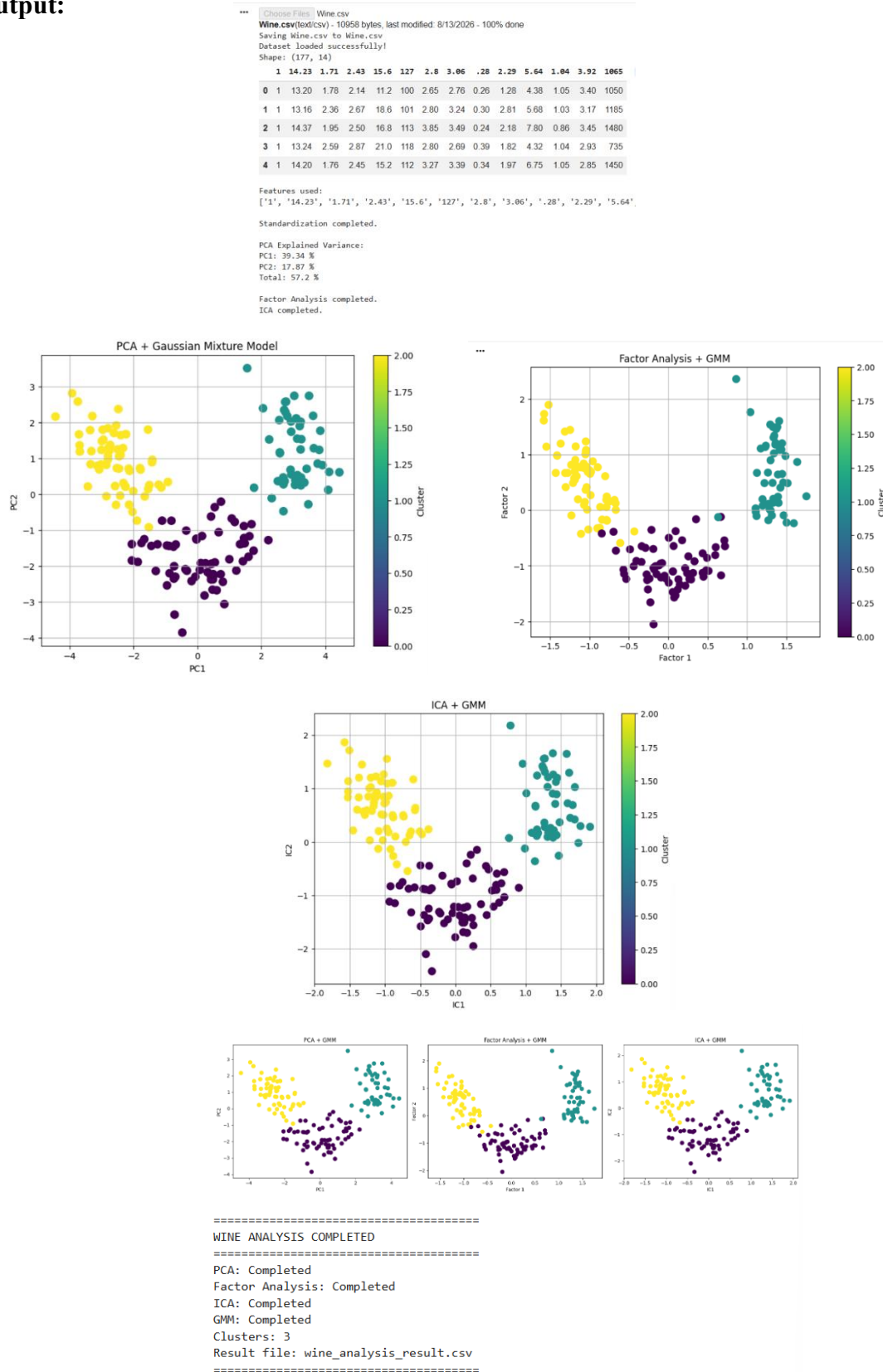
print("Result file: wine_analysis_result.csv")

print("=====")

files.download("wine_analysis_result.csv")

```

Output:

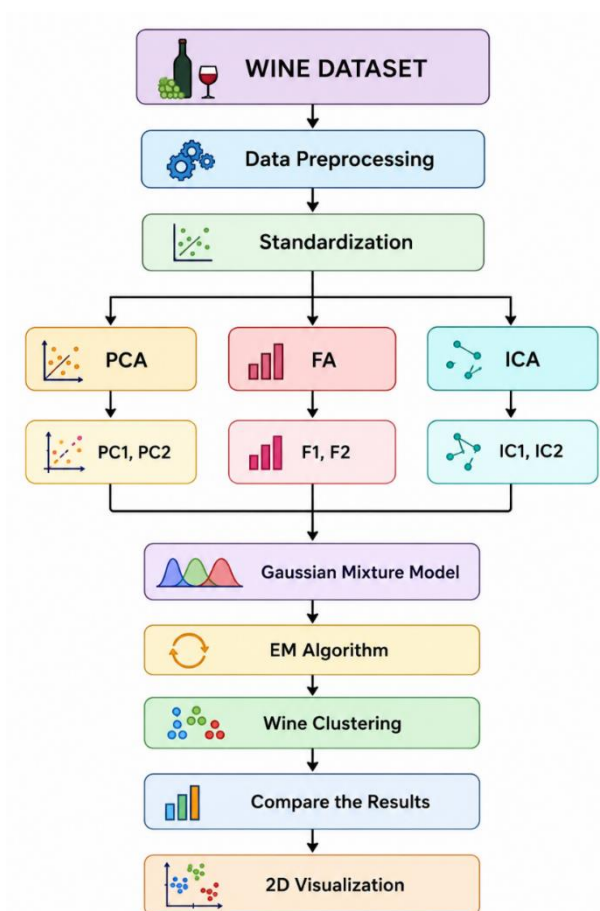


Applications

This type of analysis can be used for:

- 🍷 Wine classification
- 🧪 Chemical analysis
- 🏭 Laboratory data analysis
- 🏭 Quality control
- 🏥 Medical data analysis
- 🔍 Anomaly detection

Overall Methodology:



Conclusion:

This case study demonstrates how PCA, Factor Analysis, and ICA can reduce the dimensionality of wine chemical data. Standardization ensures that features with different scales are treated fairly. After transformation, the Gaussian Mixture Model using the EM algorithm can group wine samples into clusters.

Therefore, combining dimensionality reduction with GMM makes the wine dataset easier to analyze, visualize, and cluster.